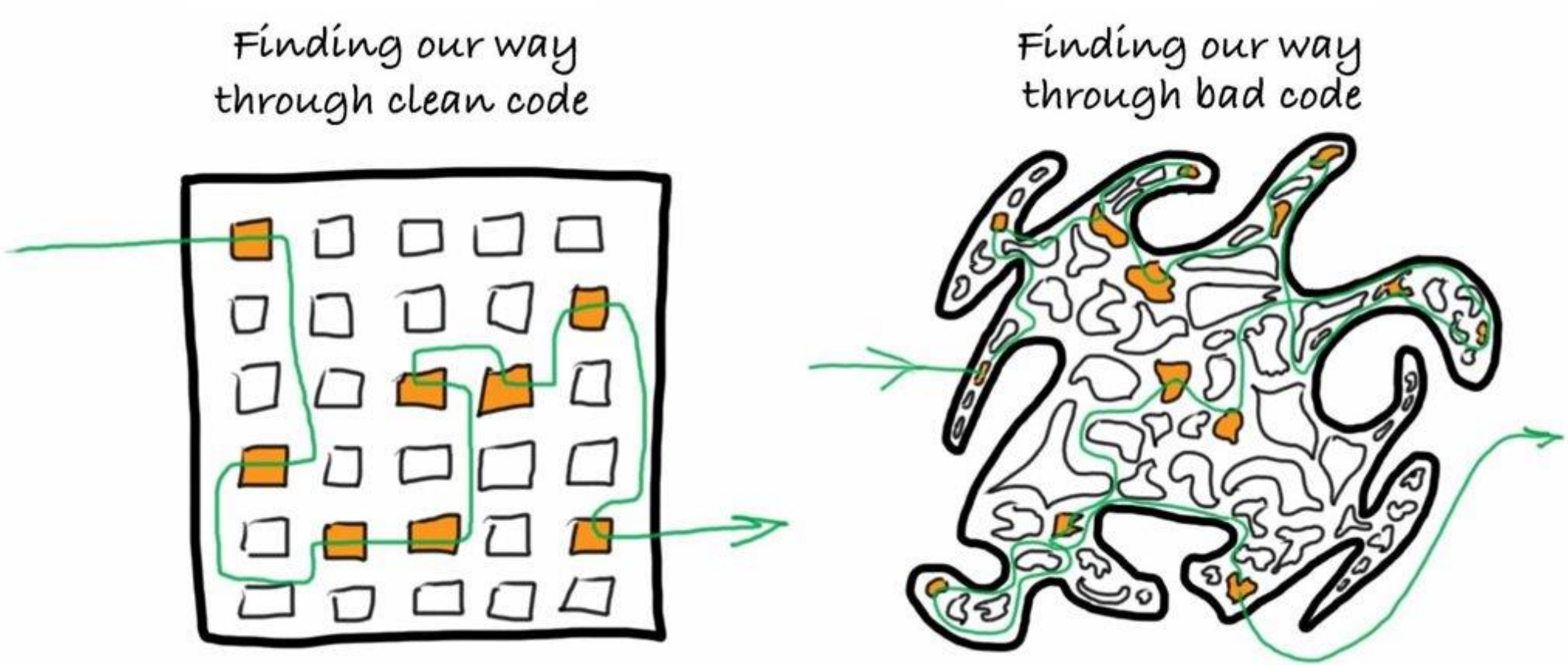
Model-View-Controller (MVC)

A pattern to organize code in software, especially for web applications like online stores, social media platforms, or booking systems.



Good vs. Bad Code



Why Does Software Need Organization?



Imagine you’re running a busy restaurant, and you’ve got chefs cooking food in the kitchen, waiters serving customers at tables, a menu that shows what’s available.

Now imagine everyone did whatever they wanted

- Chefs serving drinks,
- Waiters cooking, and
- A menus changing randomly.

The restaurant would be in total chaos!

Why Does Software Need Organization?

To keep things running smoothly, you need a system:

- Chefs handle cooking,
- Waiters manage customer interactions, and
- The menu clearly displays the options.

This division of roles is a lot like how we organize software using the Model-View-Controller (MVC) pattern.



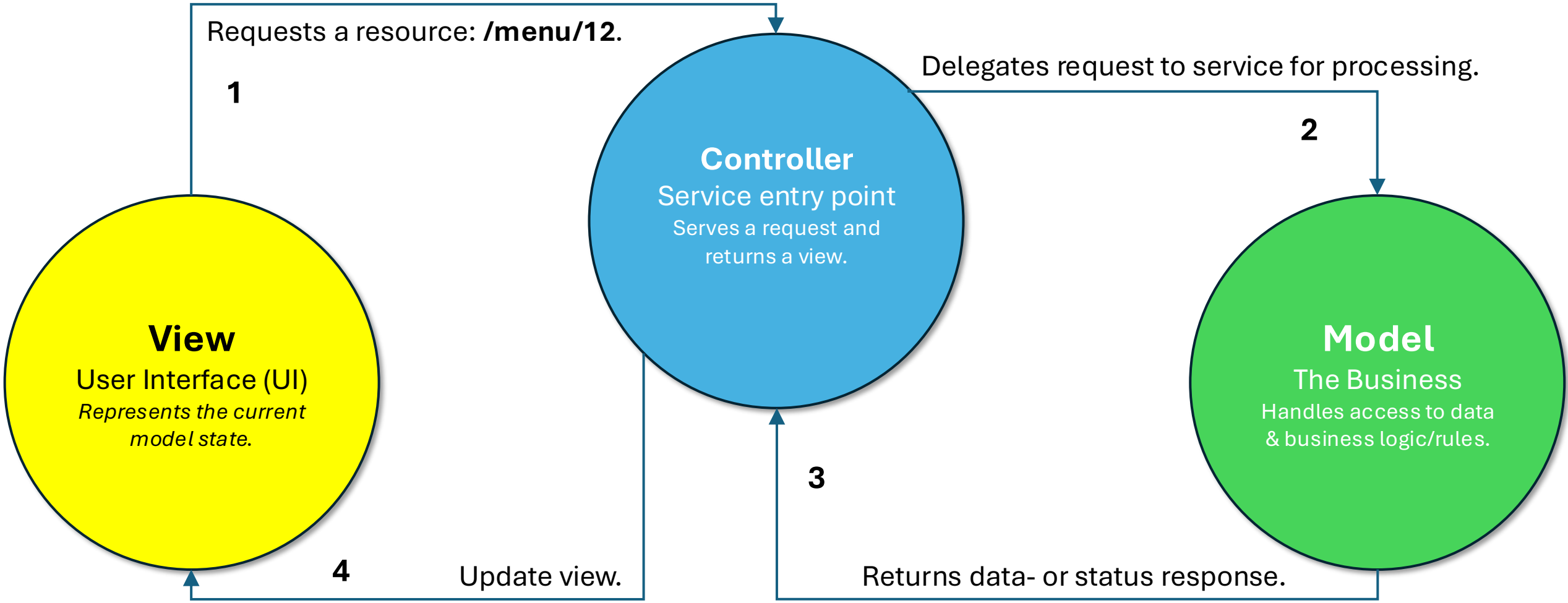
What is the MVC Pattern?

The MVC pattern is a way to organize code in software, especially for web applications like online stores, social media platforms, or booking systems.

It splits the work into three clear roles: Model, View, and Controller. Let’s break it down using our restaurant analogy.

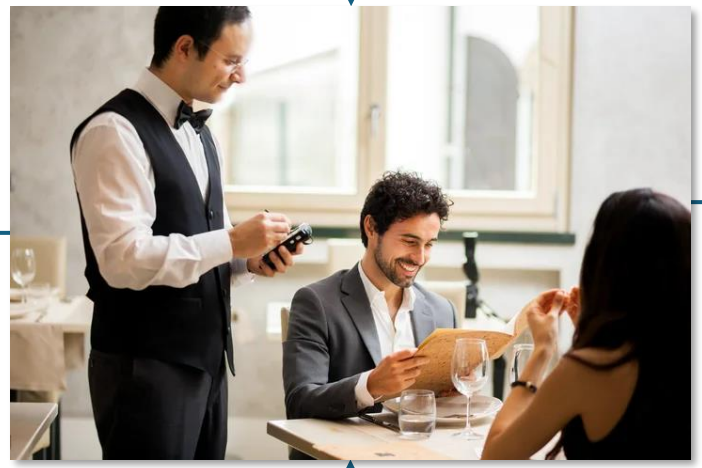
- **Model: The Kitchen (Where the Data Lives)**
- **View: The Menu and Dining Area (What the User Sees)**
- **Controller: The Waiter (The Go-Between)**

MVC Architecture Pattern



MVC Architecture Pattern

Gives order



Relays order.



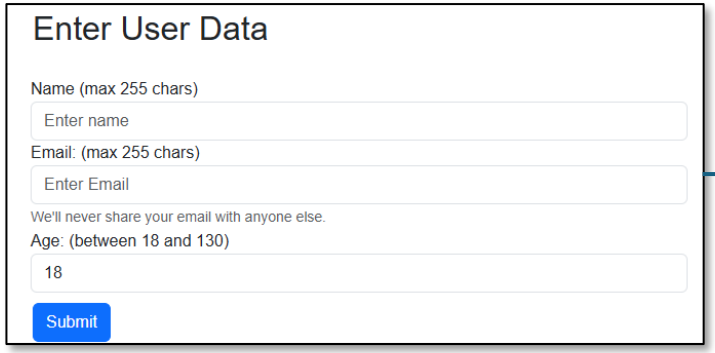
Prepares and send food.

Takes food to the right table.



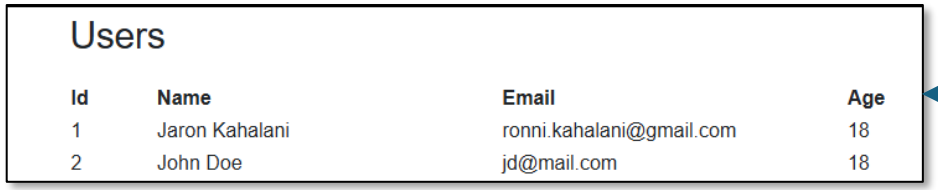
How Spring Thymeleaf Supports MVC

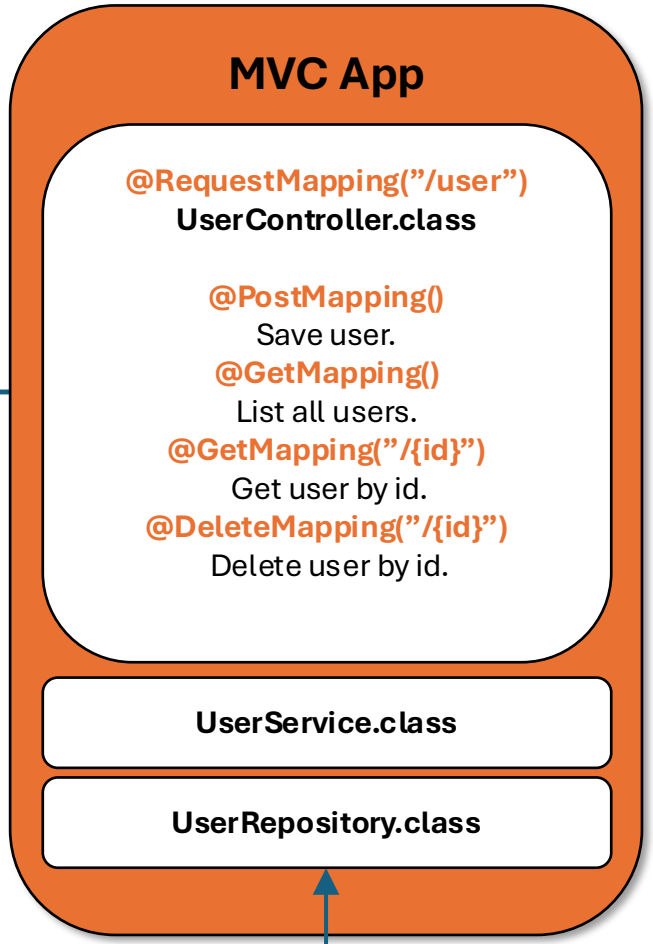
View: input.html

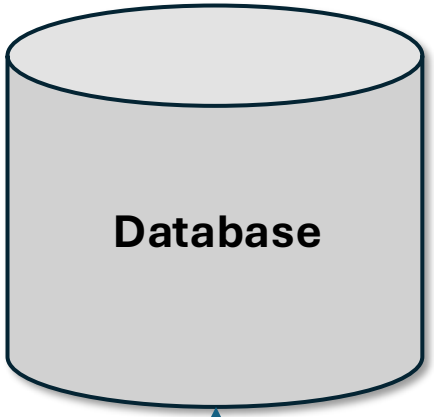


Form action
POST /submit

View: list.html







Entity CRUD + queries,
like findAll(), findById(), findByXYZ()

MVC Pattern - Model

The Kitchen (Where the Data Lives)
The Model **is like the kitchen in our restaurant**. It's where all the "ingredients" and "recipes" are stored and prepared.

In software, the Model **represents the data and the logic that manages it**.

For example, in an online store, the Model **might store information about products (like names, prices, and stock levels)** and handle tasks like updating stock when someone buys something.

The Model **doesn't care about how the data looks or how users interact with it**, it just makes sure the data is accurate and ready when needed.



MVC Pattern - View

The Menu and Dining Area (What the User Sees)
The View **is like the menu and the dining area**.

It's **what customers see and interact with**.

In a web application, the View is **the user interface**, build by the web pages, buttons, and forms **you see on your screen**.

For example, in an online store, the View **shows you product images, prices, and an "Add to Cart" button**.

The View's job is to **display the data from the Model** in a way that's clear and appealing, but it doesn't mess with the data itself.



MVC Pattern - Controller

The Waiter (The Go-Between)
The Controller **is like a waiter who takes your order** from the dining area, **passes it to the kitchen**, and **brings the food back to you**. In software, the Controller **handles user interactions**.

When you click “Add to Cart” on a website, the Controller **takes that request**, talks to the Model to update the cart, **and tells the View to show** you the updated cart page.

The Controller **keeps everything coordinated** without letting the Model and View get tangled up.



Why MVC Matters

- By splitting tasks into Model, View, and Controller, MVC keeps software organized.
- Imagine if the chef had to serve tables and design menus—things would get messy fast!
- Similarly, in software, MVC makes it easier for developers to work together, fix bugs, and add new features without breaking everything.
- It’s like having a well-run restaurant where everyone knows their role.
- And it’s easier to refactor (add and change) the structure, behavior and looks of the different MVC components.

How Spring Thymeleaf Supports MVC

- Helps developers build web applications using the MVC pattern.
- Spring is a popular framework in the programming language Java.
- Thymeleaf + Spring makes views easier to create and manage.
- Think of Spring as the restaurant’s management system, and Thymeleaf as the tool that designs beautiful, easy-to-read menus for customers.

What is Spring?

Spring is like the restaurant’s manager, it

- Provides a structure for organizing the Model, View, and Controller.
- Makes it easier for developers to handle things like processing user requests, managing data, and creating web pages.

Spring is widely used because it saves time and keeps code organized.

What is Thymeleaf?

Thymeleaf is a tool that focuses on the View part of MVC. It

- Helps developers create web pages that are dynamic, meaning they can change based on the data from the Model.
- For example, in our online bookstore, Thymeleaf helps create a page that shows different books depending on what the user searches for.

How Spring Thymeleaf Supports MVC

- **Model (The Kitchen)** - Spring manages the data and logic. For example, it might store a list of books and handle tasks like calculating the total cost of a user's cart. Spring makes it easy to connect the Model to databases or other systems where data is stored.
- **View (The Menu)** - Thymeleaf creates the web pages users see. It uses templates—think of them as reusable menu designs. A Thymeleaf template might have placeholders like "Insert book title here" or "Show price here." When the Controller sends data from the Model, Thymeleaf fills in those placeholders to create a customized page. For example, if you search for "mystery books," Thymeleaf generates a page showing only mystery books, with their titles and prices pulled from the Model.
- **Controller (The Waiter)** - Spring handles the Controller, which takes user actions (like clicking "Search" or "Add to Cart") and decides what to do. The Controller talks to the Model to get or update data, then passes that data to Thymeleaf to create the View. For instance, when you add a book to your cart, the Controller updates the cart in the Model and tells Thymeleaf to show you the updated cart page.

Why Spring Thymeleaf is Great for MVC Easy to Use

Thymeleaf templates look like regular HTML (the language of web pages), so designers and developers can work together easily.

It’s like letting a graphic designer create a beautiful menu without needing to know how the kitchen works.

- **Dynamic and Flexible**: Thymeleaf can create pages that change based on user actions, like showing different products or personalizing a page with your name.
- **Keeps MVC Clean**: Spring handles the Model and Controller, while Thymeleaf focuses on the View, keeping the roles separate and organized.
- **Reusable**: Thymeleaf templates can be reused across different pages, like using the same menu layout for breakfast and dinner but swapping out the dishes.

Why This Matters in the Real World

The MVC pattern, with tools like Spring Thymeleaf, powers many of the websites and apps you use every day

By keeping data, user interfaces, and interactions separate, MVC makes it easier to build apps that are:

- **Reliable** - Bugs in one part (like the View) won't break the whole system.
- **Scalable** - Developers can add new features, like a recommendation system for books, without starting from scratch.
- **User-Friendly** - Thymeleaf helps create clean, interactive web pages that make your experience smooth and enjoyable.

For developers, Spring Thymeleaf is like having a well-trained restaurant staff and a great menu designer.

It lets them focus on building cool features instead of wrestling with messy code.

A Quick Example

A very simple MVC app to register a user, with Spring Thymeleaf.